

[← Go Back](#)

Hardware

Nicla Vision

Tutorials

[Blob Detection with OpenMV](#)[Smart Elevator Monitoring System with the Portenta Proto Kit](#)[Getting Started with Nicla Vision](#)[Image Classification with Edge Impulse®](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Video Live Streaming with OpenMV and Nicla Vision](#)[Reading Audio Samples With the Onboard Microphone](#)[Accessing IMU Data on Nicla Vision](#)[Proximity Detection with Arduino Nicla Vision](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Nicla Vision User Manual](#)[Home](#) / [Hardware](#) / [Nicla Vision](#) /[Getting Started with Nicla Vision](#)

Getting Started with Nicla Vision

This tutorial teaches you how to get started with the computer vision capabilities of the Nicla Vision, leveraging the OpenMV IDE and MicroPython.

Benjamin Dannegård
Author & Christopher Méndez
Last revision · 12/18/2023

ON THIS PAGE

Overview

Goals	+
Introduction	+
Using the Nicla Vision Camera	
Using the Nicla Vision with Arduino IDE	
Conclusion	+
Troubleshooting	+

Overview

The OpenMV IDE is meant to provide an Arduino-like experience for simple machine vision tasks using a camera sensor. In this tutorial, you will learn about some of the basic features of the OpenMV IDE and how to create a simple MicroPython script. The Nicla Vision has OpenMV firmware on the board by default, simplifying the connection with the OpenMV IDE.



If you are willing to use the Nicla Vision with the "Arduino IDE" instead, navigate to this [section](#) for more details.

Goals

Learn how to use the basic features of the OpenMV IDE

Learn how to create a simple MicroPython script with the OpenMV IDE

Learn how to use the OpenMV

[Help](#)

Required Hardware and Software

Nicla Vision

Micro USB cable (either USB-A to Micro USB or USB-C® to Micro USB)

OpenMV IDE v4.0.10 or higher

Introduction

By using the OpenMV IDE, you can run **MicroPython** scripts on your Nicla Vision. MicroPython provides a lot of classes and modules that facilitate the testing of Nicla Vision's features.

In this tutorial, first, you will learn how to download the OpenMV IDE and set up the development environment. You can read more about the OpenMV IDE on the [official website of the project](#). OpenMV comes with its own firmware that is built in MicroPython and must be loaded inside your board before starting to create your own programs. Once your board is updated and configured, you will then learn how to write a simple script that will blink the onboard RGB LED using some basic MicroPython commands and how to create a program that uses computer vision.



Before proceeding with the tutorial, please update the board's bootloader following the instructions below.

1. Bootloader Firmware

To update the bootloader firmware of your product, please do the following steps:

Download the latest version of the board support package by searching for

Arduino Mbed OS Nicla Boards

in the Arduino IDE's

Boards Manager.



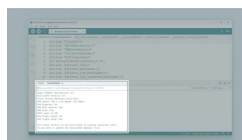
Nicla Vision board
support package
download

Then, go to **File > Examples > STM32H747_System > STM32H747_manageBootloader** and upload this sketch to your board.



Example sketch
location for
bootloader update

After the sketch is uploaded, follow the instructions in the Serial Monitor.



Serial Monitor
instructions and
current bootloader
info

If your Nicla Vision does not have the latest bootloader,

of the Serial Monitor, tap on Enter and wait for the update.

If you get the "error exit status 74", verify you close each serial monitor window and you selected the board serial port correctly before trying again.

2. Downloading the OpenMV IDE

Before you can start programming OpenMV scripts for the Nicla Vision, you need to download and install the OpenMV IDE.

Open the [OpenMV download](#) page in your browser, download the latest version available for your operating system, and follow the instructions of the installer.

3. Nicla Vision LED Codes

While using the Nicla Vision with OpenMV, the RGB LED of the board can be used to inform the user about its current status. Some of the most important ones are the following:

 **Blinking Green:** Your Nicla Vision onboard bootloader is running. The onboard bootloader runs for a few seconds when your Nicla Vision is powered via USB to allow OpenMV IDE to reprogram your Nicla Vision.

 **Blinking Blue:** Your Nicla Vision is running the default `main.py` script onboard.

If you overwrite the `main.py` script on your Nicla Vision, then it will run whatever code you loaded on it instead

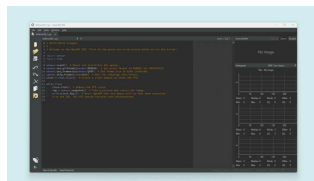
If the LED is blinking blue but OpenMV IDE cannot connect to your Nicla Vision, please make sure you are connecting your Nicla Vision to your PC with a USB cable that supplies both data and power.

Blinking White: Your Nicla Vision firmware is panicking because of a hardware failure. Please check that your Nicla Vision's camera module is installed securely.

If you tap the Nicla Vision reset button once, the board resets. If you tap it twice, the board enters in Device Firmware Upgrade (DFU) mode and its green LED starts blinking and fading.

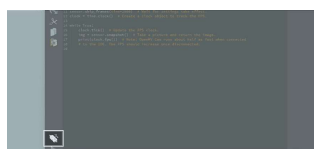
4. Connecting to the OpenMV IDE

Open the OpenMV IDE and connect the Nicla Vision to your computer via the USB cable if you have not done so yet.



The OpenMV IDE after starting it

Click on the "connect" symbol at the bottom of the left toolbar.



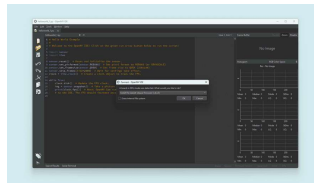
Click the connect button to attach the Nicla Vision to the OpenMV IDE

If your Nicla Vision has not the latest firmware, a pop-up will ask you to install it. Your board will enter in DFU mode and it's green LED will start fading.

Select

Install the latest [release](#) firmware

. This will install the latest OpenMV firmware on the Nicla Vision. You can leave the option of erasing the internal file system unselected and click .



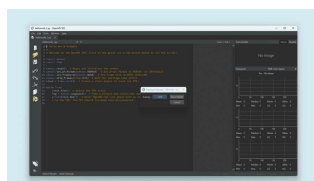
Install the latest version of the OpenMV firmware

Nicla Vision's green LED will start flashing while the OpenMV firmware is being uploaded to the board. A loading bar will start showing you the flashing progress.

Wait until the green LED stops flashing and fading. You will see a message saying

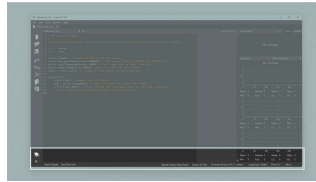
DFU firmware update complete!

when the process is done.



Installing firmware on Nicla Vision board in

The board will start flashing its blue LED when it is ready to be connected. After confirming the completion dialog, the Nicla Vision should already be connected to the OpenMV IDE, otherwise click the "connect" button (plug symbol) once again (the blue blinking should stop).



When the Nicla Vision is successfully connected a green play button appears

5. Preparing the Script

Create a new script by clicking the "New File" button in the toolbar on the left side. Import the required module `pyb`:

```
1 import pyb # Import module
```

A module in Python® is a confined bundle of functionality. By importing it into the script, it becomes available. For this example you will only need `pyb`, which is a module that contains board related functionality such as PIN handling. You can read more about its functions [here](#).

Now you can create the variables that will control our built-in RGB LED. With `pyb` you can easily control each color.

```
1 redLED = pyb.LED(1) # build
2 greenLED = pyb.LED(2) # bu
3 blueLED = pyb.LED(3) # bui
```

At this point, you can easily distinguish between which color you control in the script.

6. Creating the Main Loop in the Script

Adding the code inside a while loop will make the code run continuously.

In the loop, you can turn on the LED with `on`, then you can use the `delay` function to create a delay.

This function will start executing with the next instruction in the script. The duration of the delay can be controlled by changing the value inside the parentheses. The number defines how many milliseconds the board will wait. After the specified time has passed, you can turn off the LED with the `off` function. You can repeat that for each color.

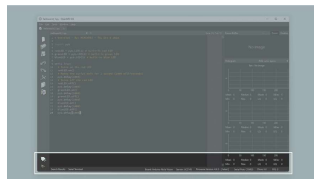
```
1 while True:
2     # Turns on the red LED
3     redLED.on()
4     # Makes the script wait
5     pyb.delay(1000)
6     # Turns off the red LED
7     redLED.off()
8     pyb.delay(1000)
9     greenLED.on()
10    pyb.delay(1000)
11    greenLED.off()
12    pyb.delay(1000)
13    blueLED.on()
14    pyb.delay(1000)
15    blueLED.off()
16    pyb.delay(1000)
```

7. Uploading the Script

Here you can see the complete blink script:

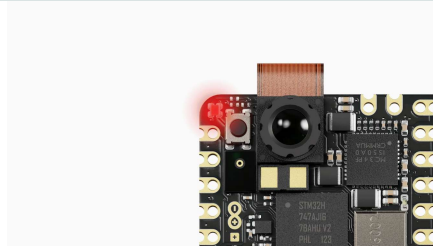
```
1 import pyb # Import modul
2
3 redLED = pyb.LED(1) # bui
4 greenLED = pyb.LED(2) # b
5 blueLED = pyb.LED(3) # bu
6
7 while True:
8
9     # Turns on the red LED
10    redLED.on()
11    # Makes the script wait
12    pyb.delay(1000)
13    # Turns off the red LED
14    redLED.off()
15    pyb.delay(1000)
16    greenLED.on()
17    pyb.delay(1000)
18    greenLED.off()
19    pyb.delay(1000)
20    blueLED.on()
21    pyb.delay(1000)
22    blueLED.off()
23    pyb.delay(1000)
```

Connect your board to the OpenMV IDE and upload the above script by pressing the play button in the lower left corner.



Press the green play button to upload the script

Now, the built-in LED on your Nicla Vision board should be blinking red, green and then blue repeatedly.



Using the Nicla Vision Camera

You can easily access the camera on the Nicla Vision through OpenMV IDE. Below is a short script that will set up the camera and take a photo. The board will blink its LED to indicate when it will take the picture.

The taken photo will be saved in the Nicla Vision drive directory and will be accessible after tapping the reset button.

```
1 import pyb # Import modul
2 import sensor # Import th
3 import image # Import mod
4
5 redLED = pyb.LED(1) # bui
6 blueLED = pyb.LED(3) # bu
7
8 sensor.reset() # Initiali
9 sensor.set_pixformat(sens
10 sensor.set_framesize(sens
11 sensor.set_vflip(True) #
12 sensor.set_hmirror(True)
13
14 redLED.on()
15 sensor.skip_frames(time =
16
17 redLED.off()
18 blueLED.on()
19
20 print("You're on camera!")
21 sensor.snapshot().save("e
22
23 blueLED.off()
24 print("Done! Reset the ca
```

The camera that comes with the Nicla Vision supports RGB 565 images.

That is why you have to use

```
sensor.set_pixformat(sensor.RGB565)
```

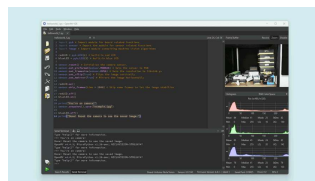
, enabling the camera to take an image with color. Then you need to set the resolution of the camera.

Here we will use

```
sensor.set_framesize(sensor.QVGA)
```

Using `sensor.set_vflip` and `sensor.set_hmirror` will help you set the correct orientation of the image. If you hold the board with the USB cable facing down, you should call `sensor.set_vflip(True)`. The image will be mirrored, if you want the image to be displayed as you see it from your perspective, you should call `sensor.set_hmirror(True)`.

Running this script in OpenMV will show the image that the camera is currently capturing in the top right corner, inside the frame buffer. The onboard red LED will be on for a couple of seconds, then the blue LED will turn on to indicate when the picture is about to be taken. A message will be printed in the serial terminal when the image is taken.



Where to see the live preview in OpenMV

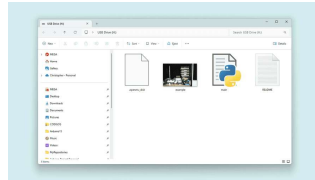
The image will be saved as

```
"example.jpg"
```

in the boards directory. It is also possible to save

the image in a `"hmn"` format. If you

reset button, the image file will appear in the boards directory.



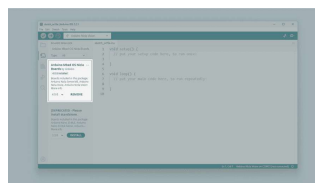
Where to find the taken picture

If you want to try the built-in OpenMV IDE examples using,  macOS, you may face format issues, use this [converter](#) if needed.

Using the Nicla Vision with Arduino IDE

As mentioned before, the Nicla Vision comes with OpenMV firmware pre-installed. This makes it easier to use the board with OpenMV out of the box. However, it is possible to use the Nicla Vision with the Arduino IDE.

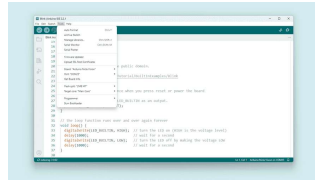
First make sure that you have the latest core installed. To install the core navigate to **Tools > Board > Boards Manager...**, in the Boards Manager window search for **Mbed OS Nicla Boards** and install it.



Nicla Vision board support package download

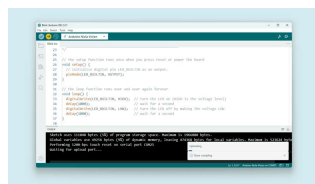
When the core is installed and you

board is connected to and the board core.



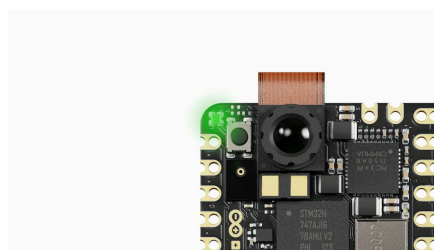
Board and port selection

Navigate to **File > Examples > 01.Basics > Blink** to open the Blink example. You should now be able to upload an Arduino sketch to the board by clicking on the "Upload" button.



Uploading the Blink example from the Arduino IDE

The board should start blinking the on-board green LED.



If you wish to use the board with OpenMV after it has been used with the Arduino IDE, you have to put the board into bootloader mode and install OpenMV firmware. You can do this by double pressing the reset button, located next to the LED. When the board is in bootloader mode and connected to your computer, follow the steps above in

section to connect the board to the OpenMV IDE again.

Conclusion

In this tutorial you learned how to use the OpenMV IDE with your Nicla Vision board. You also learned how to control the Nicla Vision's RGB LED with MicroPython functions and to upload the script to your board using the OpenMV IDE.

Next Steps

Experiment with MicroPython's capabilities. If you want some suggestion on what to do, take a look at the examples included in the OpenMV IDE. Go to: **File > Examples > Arduino >** in the OpenMV IDE.

It is possible to use the board for more advanced image processing tasks. Be sure to take a look at our other tutorials if you want to learn more.

Take a look at our other Nicla Vision tutorials which showcase its many uses. You can find them [here](#).

Troubleshooting

OpenMV Firmware Flashing Issues

If the upload of the OpenMV firmware fails during the download, put the board back in bootloader mode and try again. Repeat until the firmware gets successfully uploaded.

If the OpenMV IDE still cannot

Specific Firmware File" option.
You can find the latest firmware
in the [OpenMV Github
repository](#). Look for a file
named **firmware.bin**.

If you see a "OSError: Reset
Failed" message, reset the
board by pressing the reset
button. Wait until you see the
blue LED flashing, connect the
board to the OpenMV IDE and
try running the script again.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

